

汇编上机实践

2021

目录

- 实验报告格式
- 认识调试器
- 容易混淆指令
- 汇编程序控制结构
- 上机实验题目
- 附录

一、实验报告格式

- 实验序号
- 实验内容
- 算法或算法流程图
- 汇编程序
- 运行结果

二. 认识调试器OllyDbg/immunityDebugger

00401004	CC	INT3
00401005	\$ E9 06000000	JMP 425.00401010
0040100A	CC	INT3
0040100B	CC	INT3
0040100C	CC	INT3
0040100D	CC	INT3
0040100E	CC	INT3
0040100F	CC	INT3
00401010	> 33C0	XOR EAX,EAX
00401012	. 33D2	XOR EDX,EDX
00401014	. 33DB	XOR EBX,EBX
00401016	. 66:B8 6400	MOV AX,64
0040101A	. B3 07	MOV BL,7
0040101C	. F6F3	DIV BL
0040101E	. 6A 00	PUSH 0
00401020	. E8 05000000	CALL <JMP.&KERNEL32.ExitProcess>
00401025	CC	INT3

00401010=425.00401010

代码区

Address	Hex dump
00404000	28 40 00 00 00 00 00 00 00 00 00 00 96 40 00 00
00404010	58 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404050	00 00 00 00 00 00 00 00 CA 3A 23 77 00 00 00 00

数据段

Registers (FPU)

EAX	0000020E
ECX	00000000
EDX	00000000
EBX	00000007
ESP	0018FF88
EBP	0018FF90
ESI	00000000
EDI	00000000
EIP	0040101E 425.0040101E
C 0	ES 002B 32bit 0 (FFFFFFFF)
P 1	CS 0023 32bit 0 (FFFFFFFF)
A 0	SS 002B 32bit 0 (FFFFFFFF)
Z 1	DS 002B 32bit 0 (FFFFFFFF)
S 0	FS 0053 32bit 7FFDD000 (FFF)
T 0	GS 002B 32bit 0 (FFFFFFFF)
D 0	

CPU 寄存器

0018FF88	772386E3	鎔#w	RETURN t
0018FF8C	7FFDE000	. 幟	
0018FF90	0018FFD4	?□.	
0018FF94	7767BD99	鎔cw	RETURN t
0018FF98	7FFD		
0018FF9C	32BB		
0018FFA0	00000000		

堆栈段

代码区

线性地址	指令机器码	指令汇编代码
00401004	CC	INT3
00401005	\$ E9 06000000	JMP 425.00401010
0040100A	CC	INT3
0040100B	CC	INT3
0040100C	CC	INT3
0040100D	CC	INT3
0040100E	CC	INT3
0040100F	CC	INT3
00401010	> 33C0	XOR EAX,EAX
00401012	. 33D2	XOR EDX,EDX
00401014	. 33DB	XOR EBX,EBX
00401016	. 66:B8 6400	MOV AX,64
0040101A	. B3 07	MOV BL,7
0040101C	. F6F3	DIV BL
0040101E	. 6A 00	PUSH 0
00401020	. E8 05000000	CALL <JMP.&KERNEL32.ExitProcess>
00401025	CC	INT3

数据段

线性地址

数据段中数据

Address	Hex dump
00404000	28 40 00 00 00 00 00 00 00 00 00 00 00 96 40 00 00
00404010	58 40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404020	00 00 00 00 00 00 00 00 00 88 40 00 00 00 00 00 00
00404030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00404050	00 00 00 00 00 00 00 00 00 CA 3A 23 77 00 00 00 00

CPU寄存器

通用寄存器

Registers (FPU)

EAX 0000020E
ECX 00000000
EDX 00000000
EBX 00000007
ESP 0018FF88
EBP 0018FF90
ESI 00000000
EDI 00000000

指令指针寄存器

EIP 0040101E 425.0040101E

标志寄存器

C 0 ES 002B 32bit 0 (FFFFFFFF)
P 1 CS 0023 32bit 0 (FFFFFFFF)
A 0 SS 002B 32bit 0 (FFFFFFFF)
Z 1 DS 002B 32bit 0 (FFFFFFFF)
S 0 FS 0053 32bit 7FFDD000 (FFF)
T 0 GS 002B 32bit 0 (FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00000246 (NO,NB,E,BE,NS,PE,GE,LE)

ST0 empty g
ST1 empty g
ST2 empty g

堆栈段

堆栈寄存器

栈顶

堆栈段

Registers (FPU)

EAX 0000020E
ECX 00000000
EDX 00000000
EBX 00000007
ESP 0018FF88
EBP 0018FF90

0018FF88	772386E3
0018FF8C	7FFDE000
0018FF90	0018FFD4
0018FF94	7767BD99
0018FF98	7FFDE000
0018FF9C	32BBCE4C
0018FFA0	00000000
0018FFA4	00000000
0018FFA8	7FFDE000
0018FFAC	C9D49CFF
0018FFB0	FFFFFF800
0018FFB4	00000000
0018FFB8	00000000
0018FFBC	0018FF9C
0018FFC0	FFFFFFA80
0018FFC4	0018FFE4
0018FFC8	77625191
0018FFCC	45C5D410

三、容易混淆指令

除法指令

➤ 除数是8位寄存器:

被除数是16位寄存器AX

指令执行完

商存放在AL

余数存放在AH

容易混淆指令（续）

除法指令

➤ 除数是16位寄存器：

被除数是16位寄存器DX与16位寄存器AX组成：

$$(DX) * 2^{16} + (AX)$$

指令执行完

商存放在AX

余数存放在DX

容易混淆指令（续）

除法指令

➤ 除数是32位寄存器：

被除数是32位寄存器EDX与32位寄存器AX组成：

$$(EDX) * 2^{32} + (EAX)$$

指令执行完

商存放在EAX

余数存放在EDX

容易混淆指令（续）

TEST指令

➤ Test op1, op2

执行操作 op1 and op2, 并根据执行的结果是否为0,
设置cpu中标志寄存器ZF位(即Z位)

四. 汇编程序控制结构

1. 顺序结构
2. 分支结构
3. 循环结构
4. 子程序、函数调用与返回

1.3.3 X86源程序框架

```
;eg0000.asm in Windows Console
include irvine32.inc           ;包含32位输入输出文件
常量1 equ    xxxx;           ;根据需要定义常量，可以没有
常量2 equ    xxxx;
.data                          ;定义数据段
    org XX                     ;指定变量的存储偏移量，可以没有
    .....                     ;数据定义（数据待填）
.code                          ;定义代码段
main proc                     ;主过程开始
    .....                     ;主程序（指令待填）
    exit                       ;程序正常执行结束
main endp                     ;主过程结束
    .....                     ;子过程（指令待填）
end main                       ;汇编结束
```

;通常从主过程的第一条指令开始执行



X86源程序框架

---包含伪指令 INCLUDE

1. 将常用的常量定义、过程说明、共享的子程序库等内容进行声明（相当于C和C++语言中，包含头文件的作用）
2. **IRVINE32.INC是教学用的开发工具**的包含文件
 - 对用到的一些库函数做了原型声明
 - 包含了SmallWin.inc

X86源程序框架

---段的简化定义

1. 数据段定义伪指令`.DATA`创建一个数据段
2. 代码段定义伪指令`.CODE`创建一个代码段
3. 堆栈段由Windows维护，用户不必设置
4. 程序开始执行的位置
 - 应用一个标号（例如：START）或主过程main
 - 汇编结束END指令的参数
5. 应用程序执行结束
 - 语句“EXIT”将控制权交还操作系统
6. 源程序汇编结束END语句

执行结束 ≠ 汇编结束

X86源程序框架

---〔例〕 Hello, world!信息显示程序

1. 在数据段给出这个字符串形式的信息:

;数据段

```
msg byte 'Hello, world!',13,10,0
```

;定义要显示的字符串

2. 在代码段编写显示字符串的程序:

;代码段

```
mov edx,offset msg
```

;edx指定字符串的偏移地址

```
call writestring
```

;调用I/O子程序显示信息

源程序框架

---X86汇编语言源程序

;eg0000.asm in Windows Console

include irvine32.inc

.data ;数据段

msg byte 'Hello, world!',13,10,0

.code ;代码段

main proc ;主过程定义

mov edx,offset msg

call writestring

exit ;程序正常执行结束

main endp

end main ;汇编结束
运行结果

Hello, world !

源程序框架

——C源程序

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello, world !\n");
```

```
    exit(0);
```

```
}
```

运行结果

Hello, world !

源程序框架

---ARM源程序

```
.text
.global tart1
tart1:
    mov x0,#0
    ldr x1,=msg
    mov x2,len
    mov x8,64
    svc #0

    mov x0,123
    mov x8,93
    svc #0

.data
msg:
    .ascii "Hello World!\n"
len=.-msg
```

运行结果

Hello world !

信息显示程序

汇编语言程序

——X86汇编语言源程序

```
;eg0000.asm in Windows Console
include irvine32.inc
.data                                ;数据段
msg      byte 'Hello, world!',13,10,0
.code                                ;代码段
main proc                            ;主过程定义
    mov  edx,offset msg
    call writestring
    exit                                ;程序正常执行结束
main endp
end main                            ;汇编结束
```

C语言程序

```
#include <stdio.h>
int main()
{
    printf("Hello, world !\n");
    exit(0);
}
```